
Algoritmi e programmazione

Di cosa parleremo:

01 Che cos'è un algoritmo

03 Lo pseudocodice

02 Il diagramma di flusso

04 Linguaggi di programmazione

01

Che cos'è un algoritmo

Una sequenza finita di passi che trasforma un input in un output.

Muḥammad ibn Mūsā al-Khwārizmī

🌐 150 lingue ▼

Voce [Discussione](#)

Leggi [Modifica](#) [Modifica wikitesto](#) [Cronologia](#) ⋮

Abū Jaʿfar Muḥammad ibn Mūsā al-Khwārizmī (in [persiano](#) جعفر محمد بن موسی خوارزمی

محمد بن موسی خوارزمی, *Abu Jaʿfar Moḥammed ebn Musā*

Khwārezmi, in [arabo](#) ابو جعفر محمد بن موسی الخوارزمي^[?] ;

[Corasmia](#), 780 circa – 850 circa) è stato un [matematico](#), [astronomo](#) e [geografo persiano](#), conosciuto come il padre dell'[algebra](#).^{[1][2][3][4]}

Nativo - come dice la sua [nisba](#) - della regione centroasiatica del [Khwārezm](#) (in persiano *Kh^wārazm*, l'antica [Corasmia](#))^[5], talvolta confuso con [Jaʿfar Muḥammad ibn Mūsā ibn Shākir](#)^[6], visse a [Baghdad](#) presso la corte del califfo [al-Maʿmūn](#), che lo nominò responsabile della sua biblioteca, la famosa [Bayt al-Ḥikma](#), la Casa della sapienza, di Baghdad.

Sotto la sua direzione furono tradotte in arabo molte delle principali opere matematiche del periodo greco-ellenistico, dell'antica Persia, di [Babilonia](#) e dell'[India](#). Il cratere lunare [Al-Khwarizmi](#) è dedicato a lui, come pure l'[asteroide 11156 Al-Khwarismi](#). Il suo nome ha dato origine al termine [algoritmo](#) e la sua opera al termine [algebra](#).^{[7][8]}



Un [francobollo commemorativo](#) del 1200° anniversario (approssimativo) della nascita del grande matematico persiano, stampato in [Unione Sovietica](#) il 6 settembre 1983.

L'al-Kitāb al-mukhtaṣar fī ḥisāb al-jabr wa al-muqābala [[modifica](#) |

[modifica wikitesto](#)]

È l'autore dell'*al-Kitāb al-mukhtaṣar fī ḥisāb al-jabr wa al-muqābala*. Il trattato fu reperito nella penisola iberica dall'inglese [Roberto di Chester](#), vissuto nel XII secolo, che ne tradusse una parte in latino con il titolo *Liber algebrae et almucabala*, dove *algebrae* fu la latinizzazione di *al-jabr*. La prima traduzione completa fu opera di [Gerardo da Cremona](#).

Definizione di Algoritmo

**Una sequenza finita di passi,
da input a output**



Una procedura di calcolo ben definita che prende un valore (o un insieme di valori) come input e produce un output in un tempo finito.

Gli algoritmi sono ovunque

RICETTA

Cucinare un piatto

Dagli ingredienti (input), una sequenza di passi produce il piatto (output).

ISTRUZIONI

Montare un mobile

Passi ordinati che, seguiti alla lettera, portano al mobile finito.

PROCEDURA

Prelevare al bancomat

Inserisci carta, PIN, importo: una procedura fissa, uguale per tutti.

Algoritmo vs pseudocodice vs programma

Algoritmo

Il procedimento, indipendente dal linguaggio. Il "come si fa".



Pseudocodice

Lo descrive in modo semi-formale, tralasciando i dettagli tecnici.



Programma

L'implementazione precisa in un linguaggio: un risultato esatto per ogni input.

Rispondono alla stessa domanda "cosa fare?", descritto con precisione crescente.

Algoritmo “ben fatto”

Finitezza

termina dopo un numero limitato di passi

Eseguibilità

ogni azione è compibile in tempo finito con le risorse date

Non ambiguità

ogni istruzione si interpreta in un solo modo

Determinismo

a parità di input, sempre lo stesso output

Generalità

vale per ogni caso della classe di problemi

02

Il diagramma di flusso

Una *mappa visiva* dell'algoritmo, con pochi simboli standard.

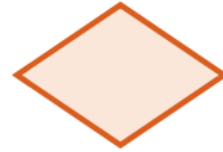
I simboli standard del diagramma di flusso:



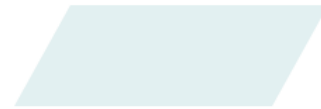
inizio / fine



azione



decisione

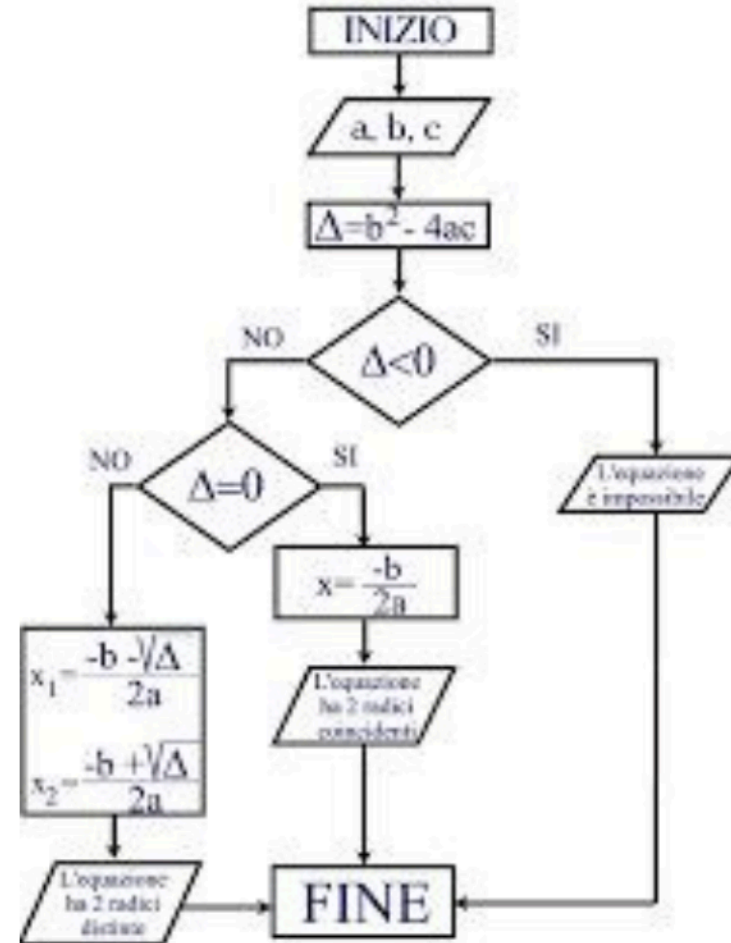
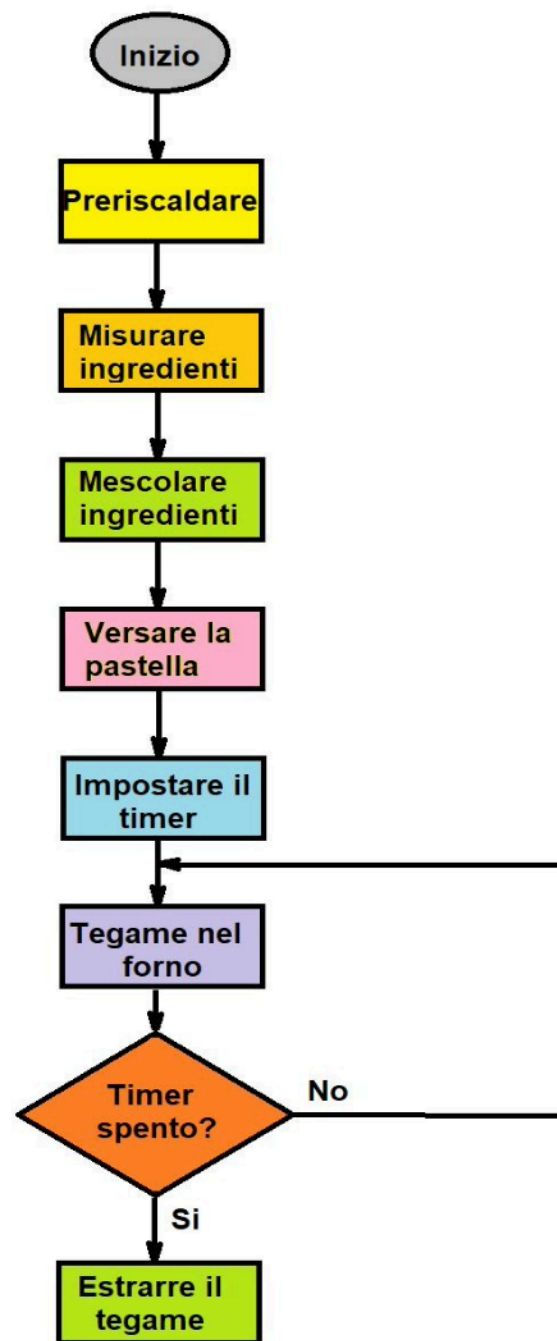


ingresso / uscita



flusso

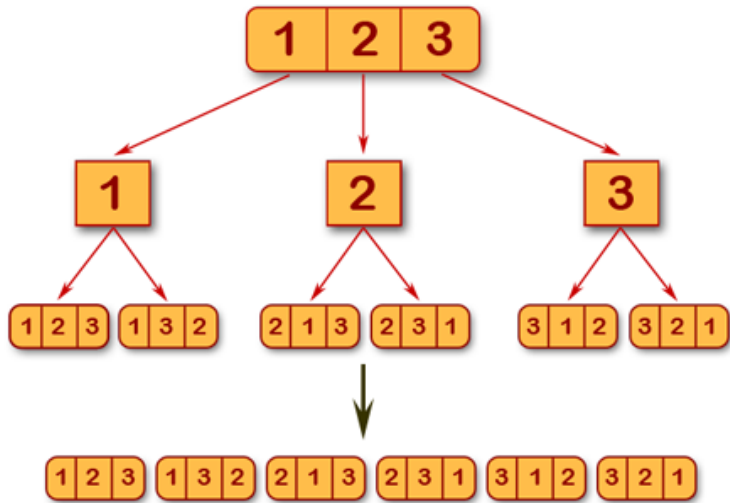
Ogni operazione è un blocco; le frecce mostrano l'ordine. Si legge dall'inizio, seguendo le frecce.



Di ogni algoritmo ci chiediamo...

Correttezza

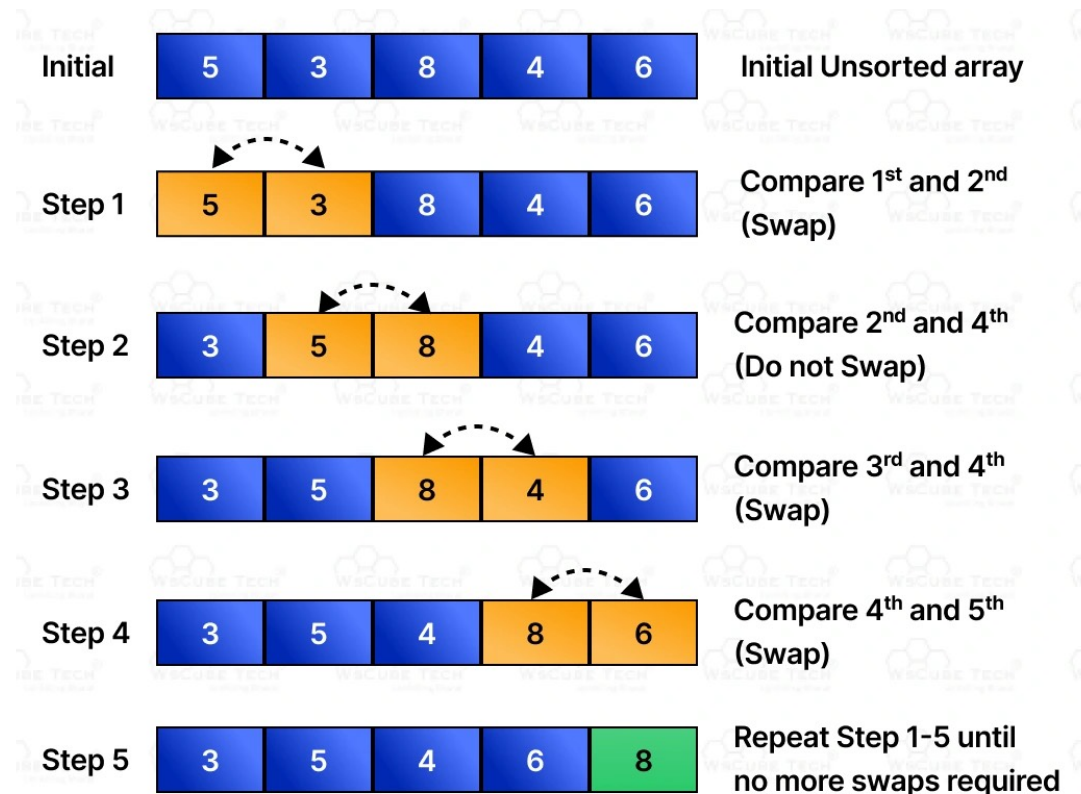
Funziona con **ogni** input possibile, sempre?



Vs

Efficienza

Quanti passi servono al **crescere** della dimensione del problema?



Dall'idea al programma funzionante

1

Specifica

definire con precisione il problema

2

Algoritmo

individuare un procedimento risolutivo

3

Codifica

tradurlo in un linguaggio

4

Esecuzione

eseguire e mettere a punto (debug)

03

Lo Pseudocodice

Cos'è lo pseudocodice

Una descrizione ad alto livello, senza dettagli implementativi

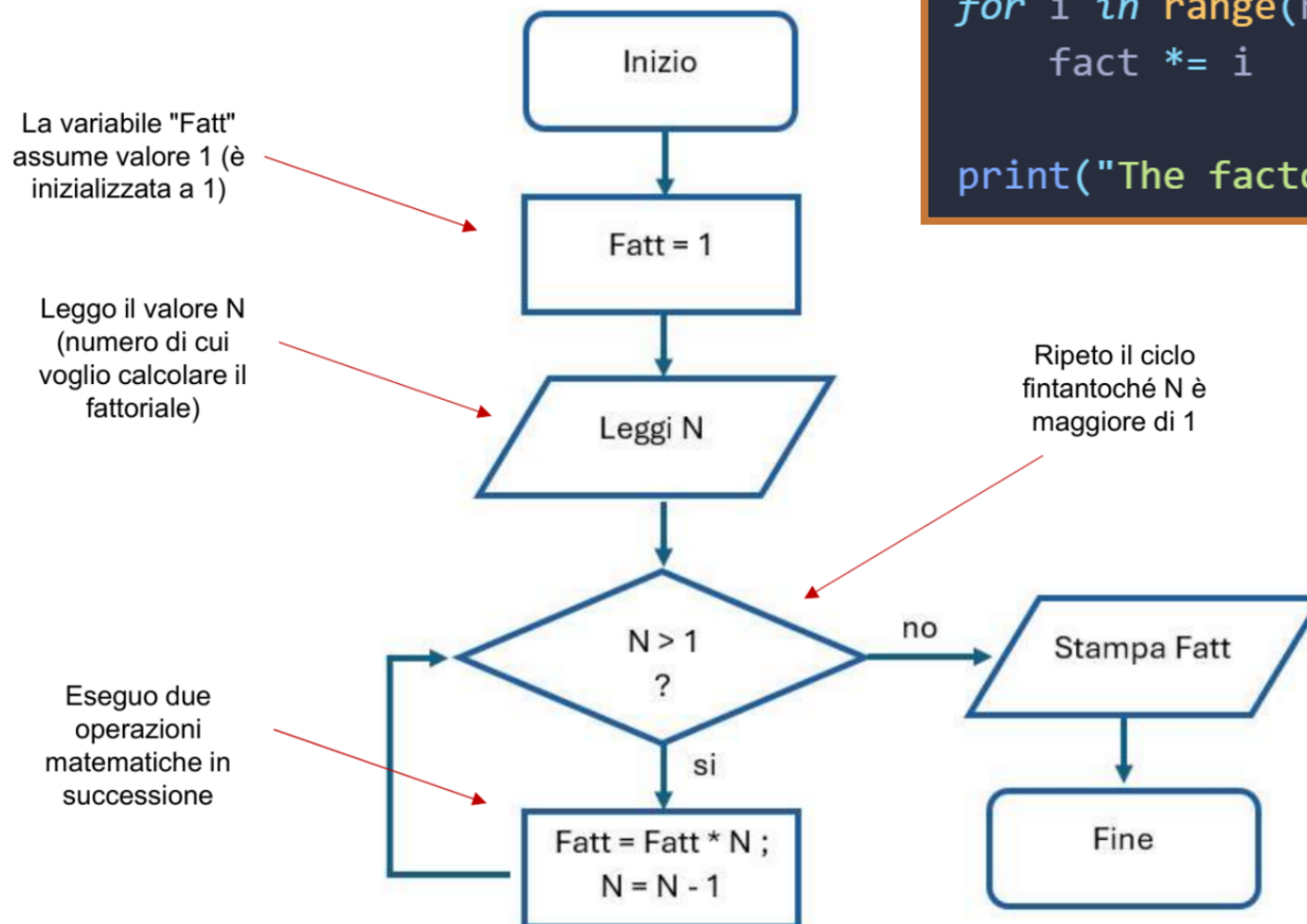
Ignora i dettagli di un linguaggio preciso, ma deve essere completa:
descrivere l'intero procedimento in modo poi traducibile in codice vero.

La sintassi dello pseudocodice non è universale.

```
IF  $n \geq 0$  THEN  
   $x = 1$   
  DOFOR  $i = 1, n$   
     $x = x \cdot i$   
  END DO  
   $fac = x$ 
```

Calcoliamo il fattoriale di un numero:

```
IF n ≥ 0 THEN  
  x = 1  
  DOFOR i = 1, n  
    x = x · i  
  END DO  
  fac = x
```



```
n = int(input("Enter number: "))  
fact = 1  
  
for i in range(n,0,-1):  
    fact *= i  
  
print("The factorial is:",fact)
```

Variabili, costanti, espressioni

variabile	un'entità che può cambiare valore (la temperatura)
costante	un valore che non cambia (4)
espressione	variabili, costanti e operatori ($x*5 + y$)
espressione booleana	vale VERO o FALSO ($n > 3$)
istruzione	una combinazione di espressioni (SET x AS $x+3$)

Altri comandi

Input / Output

READ • DISPLAY

Calcolo

SET • COMPUTE • ADD • SUBTRACT

Controllo di flusso

IF-THEN • IF-THEN-ELSE

Cicli

WHILE • DO-WHILE • REPEAT-UNTIL • FOR

Confronti / booleane

< ≤ > ≥ is ≠ • NOT AND OR

Esempio 1

Pari o dispari

```
# input: intero – output: pari/dispari  
READ numero  
IF numero % 2 is 0 THEN  
    DISPLAY "pari"  
ELSE  
    DISPLAY "dispari"  
ENDIF
```

Casi di test

[-2] → pari

[0] → pari

[3] → dispari

Il modulo % dà il resto: se il resto per 2 è 0, è pari.

Esempio 2

Il prezzo di una torta

```
READ misura
IF misura ≤ 0 THEN
    DISPLAY "errore"
ELSE IF misura < 6 THEN
    SET prezzo TO 10
ELSE IF misura ≤ 12 THEN
    SET prezzo TO 15
ELSE
    SET prezzo TO 20
ENDIF
```

Casi di test

[-3] → errore

[5] → 10

[6] → 15 **confine**

[12] → 15 **confine**

[13] → 20

I test devono includere le **condizioni di confine** (6 e 12).

Esempio 3

Tutti i numeri pari da 1 a 100

```
SET num TO 2  
WHILE num ≤ 100  
    DISPLAY num  
    ADD 2 TO num  
END WHILE
```

Con WHILE si ripete finché la condizione resta vera.

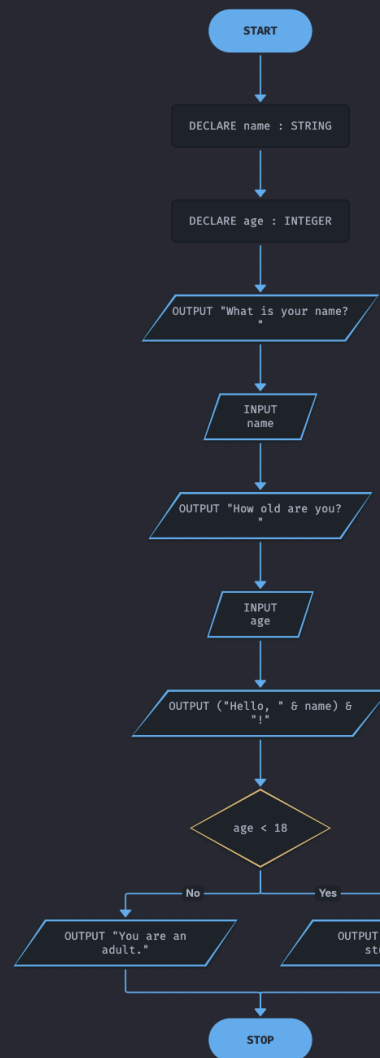
main.pseudo Open Export Debug Run

Ctrl+Enter run Ctrl+K command palette Ctrl+Shift+K stop

```

1  // Welcome! Press Run (or Ctrl+Enter) to execute this
   code.
2
3  DECLARE name : STRING
4  DECLARE age  : INTEGER
5
6  OUTPUT "What is your name? "
7  INPUT  name
8
9  OUTPUT "How old are you? "
10 INPUT  age
11
12 OUTPUT "Hello, " & name & "!"
13
14 IF age < 18 THEN
15     OUTPUT "You are a student."
16 ELSE
17     OUTPUT "You are an adult."
18 ENDIF
    
```

Terminal Trace Python Flowchart

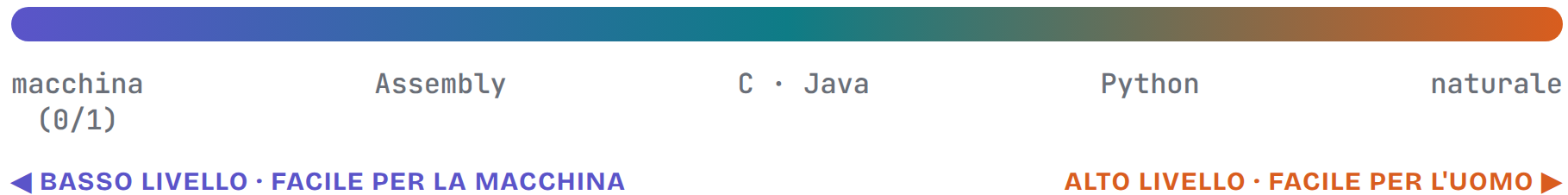


<https://pseudocode-compiler.sherlemious.com>

04

Linguaggi di programmazione

Dal linguaggio macchina al linguaggio naturale



L'Assembly, forma simbolica del codice macchina

Una rappresentazione *leggibile* delle istruzioni macchina.

Un *assembler* la traduce in 0 e 1.

macchina → Assembly

4889E5 → **mov** rbp, rsp

Perché non scrivere in linguaggio macchina?

È a bassissimo livello, difficile e pieno di trappole, e ogni architettura ha un linguaggio macchina diverso (x86-64 ≠ ARM64).

Un programma ad alto livello si può invece **ricompilare** su architetture diverse.

Compilato, interpretato, ibrido



Compilatore contro interprete

Metafora del libro

Il **compilatore** prende il libro e ne prepara una traduzione completa, da rileggere quante volte vuoi.

L'**interprete** lo legge ad alta voce traducendo sul momento, riga per riga.

Compilazione

+ veloce, efficiente, codice protetto · – meno flessibile

C, C++, Pascal, Fortran

Interpretazione

+ rapida da testare · – esecuzione più lenta

BASIC, PHP, Bash; ibridi: Python, Java